

AI Support Report

MARCO BRESCIA
GIUSEPPE GAMBARDELLA

LittleButterfly Compiler

1 AI Support

Lo sviluppo del progetto *LittleButterfly* è stato affiancato dall'utilizzo di strumenti di intelligenza artificiale basati su Large Language Models (LLM), in particolare *ChatGPT* e *Gemini*. Tali strumenti sono stati impiegati come supporto allo sviluppo e alla documentazione, senza mai sostituire il lavoro progettuale, implementativo e decisionale, che è rimasto interamente a carico degli autori. L'intelligenza artificiale è stata fondamentale per l'apprendimento iniziale del linguaggio C++ e degli strumenti per il front-end (*Flex* e *Bison*), agendo come consulente per chiarire concetti sintattici e semantici. Inoltre, ha fornito supporto nella complessa configurazione dell'infrastruttura *MLIR* e nella comprensione dei meccanismi di generazione del codice intermedio.

1.0.1 Trade-off e Criticità nell'uso dell'IA

L'adozione di questi strumenti ha evidenziato un netto **trade-off**. Da un lato, la **velocità di implementazione** è aumentata drasticamente, permettendo di superare rapidamente blocchi tecnici standard. Inoltre, si sono dimostrati essenziali per la spiegazione di codici di errore la cui risoluzione avrebbe richiesto un tempo maggiore. Dall'altro, sono emerse limitazioni significative in contesti altamente specifici o di nicchia (come errori oscuri di linking o versioni specifiche di MLIR). In diverse occasioni, i modelli tendevano a entrare in *loop logici*, proponendo reiteratamente la stessa soluzione errata per un dato problema, ignorando i feedback di fallimento forniti.

1.0.2 Strategie di Prompting

Per mitigare queste problematiche e guidare l'IA fuori da vicoli ciechi, sono state adottate tecniche di *Prompt Engineering* mirate:

- **Zero-shot prompting:** Utilizzato per spiegazioni concettuali o generazione di codice boilerplate semplice.

- **Few-shot prompting:** Fondamentale per la generazione di codice MLIR corretto. Fornendo al modello esempi concreti di codice funzionante o pattern specifici del progetto prima della richiesta, è stato possibile ottenere risposte molto più precise e contestualizzate, riducendo drasticamente le "allucinazioni" del modello.

1.0.3 Autocompletamento Intelligente

Parallelamente ai modelli conversazionali, è stato fatto ampio uso di strumenti di *inline suggestion* (come **GitHub Copilot**). L'integrazione diretta nell'ambiente di sviluppo (IDE) ha permesso di accelerare drasticamente la stesura di codice. Grazie alla capacità di prevedere interi blocchi di codice basandosi sul contesto, è stato possibile evitare la riscrittura manuale di strutture sintattiche identiche, permettendo agli sviluppatori di concentrarsi esclusivamente sulla modifica dei dettagli specifici e delle logiche divergenti.

1.0.4 Generazione dei Test e supporto alla documentazione

Un contributo essenziale ha riguardato la **generazione automatizzata dei test**. L'IA è stata impiegata per creare rapidamente suite di test diversificate, includendo *edge cases*, sintassi errate per verificare il parser e logiche di controllo complesse. Questo ha permesso di validare il compilatore con una copertura molto più ampia di quella che sarebbe stata possibile scrivere manualmente nello stesso tempo.

Infine, gli strumenti di intelligenza artificiale sono stati impiegati come ausilio nella stesura e revisione del presente documento, migliorando la chiarezza espositiva e la coerenza terminologica, pur mantenendo il controllo umano sui contenuti tecnici.